

Informatik
Hochschule Luzern (Schweiz)

Architekturdokumentation

Skitourenverwaltung & 3x3 Sicherheitsmatrix (arc42 v8)

WEBLAB

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

von

Colin Felber

von

Luzern (Schweiz)

Inhaltsverzeichnis

1 Introduction & Goals	2
1.1 Qualitätsziele	2
1.2 Stakeholder	2
2 Context & Scope	3
2.1 Fachlicher Kontext	3
2.2 Technischer Kontext	3
2.3 In Scope	3
2.4 Out of Scope	4
3 Solution Strategy	5
4 Building Block View	6
4.1 Bausteinsicht Level 1: Gesamtsystem Whitebox	6
4.2 Bausteinsicht Level 2: Frontend-Architektur	7
4.3 Bausteinsicht Level 2: Backend-Architektur	7
5 Runtime View	9
5.1 Szenario 1: Registrierung & Login	9
5.2 Szenario 2: Tour erstellen mit 3x3 Sicherheitsmatrix	9
5.3 Szenario 3: Tour-Beitritt & Notfallkontakt-Abfrage	10
6 Deployment View	12
6.1 Container & Multi-Stage Builds	12
7 Crosscutting Concepts	13
7.1 Ownership, Datenschutz & Sicherheit	13
7.2 Datenmodell & Konsistenz	13
7.3 Frontend-Konzepte & Reaktivität	14
7.4 Testing	14
8 Architekturentscheidungen (ADRs)	15
9 Qualitätsanforderungen	16
10 Risiken & technische Schulden	17

1 Introduction & Goals

Wer eine Skitour für eine Gruppe leitet, trägt viel Verantwortung. Kommt es zu einem Notfall am Berg, muss die Tourenleitung sofort die Notfallkontakte der Teilnehmenden griffbereit haben. Zudem fordert die Rechtsprechung im Ernstfall den Nachweis einer sorgfältigen Tourenplanung nach anerkannten Standards, insbesondere der 3x3-Sicherheitsmatrix (Verhältnisse, Gelände, Mensch).

Normale Chat-Apps oder Vereins-Tools reichen dafür nicht aus:

Notfallkontakte fehlen oder sind unauffindbar, und die 3x3-Planung kann nirgends strukturiert festgehalten werden. Die vorliegende Webapplikation schliesst diese Lücke.

1.1 Qualitätsziele

Prio	Qualitätsziel	Bedeutung für die Applikation
1	Datenschutz & Vertraulichkeit	Notfallkontakte sind hochsensibel. Ausschliesslich autorisierte Tourenleiter der jeweiligen Tour dürfen sie einsehen (Principle of Least Privilege).
2	Schnelligkeit & Usability	Schnelle Bedienung am Handy und Desktop. Die 3x3-Matrix ist in wenigen Minuten ausgefüllt, Notfallkontakte sind mit einem Klick sichtbar.
3	Wartbarkeit & Testbarkeit	Klare Entkopplung: Smart/Dumb-Muster im Frontend, modularer Monolith mit NestJS im Backend, abgesichert durch Unit- und Cypress-E2E-Tests.
4	Ein-Befehl-Start	Vollständiger Start des Stacks via <code>docker compose up --build</code> ohne manuelle Vorabinstallation von Node oder Datenbanken.

1.2 Stakeholder

Rolle	Wer ist das?	Erwartungshaltung
Tourenleiter/in	Leiter/in von SAC- oder privaten Touren	Strukturierte 3x3-Dokumentation zur rechtlichen Absicherung, Teilnehmerübersicht und Notfallkontakte im Ernstfall.
Teilnehmende	Mitglieder der Tourengruppe	Einfacher Beitritt per Link, Übersicht zu Treffpunkt, Route und Ausrüstung, garantierter Schutz der Notfalldaten.
Dozent / Experte	Dominik Witschard (HSLU)	Klare Softwarearchitektur, nachvollziehbar begründete Technologiewahl, Testabdeckung und verständlicher Code.

2 Context & Scope

2.1 Fachlicher Kontext

Das System fokussiert sich auf die Planung, Durchführung und rechtliche Absicherung von Touren. Externe Daten (SLF-Lawinenbulletin, Wetterdaten) werden vom Tourenleiter manuell in die 3x3-Matrix übertragen. GPX-Routentracks können als Datei hochgeladen werden.

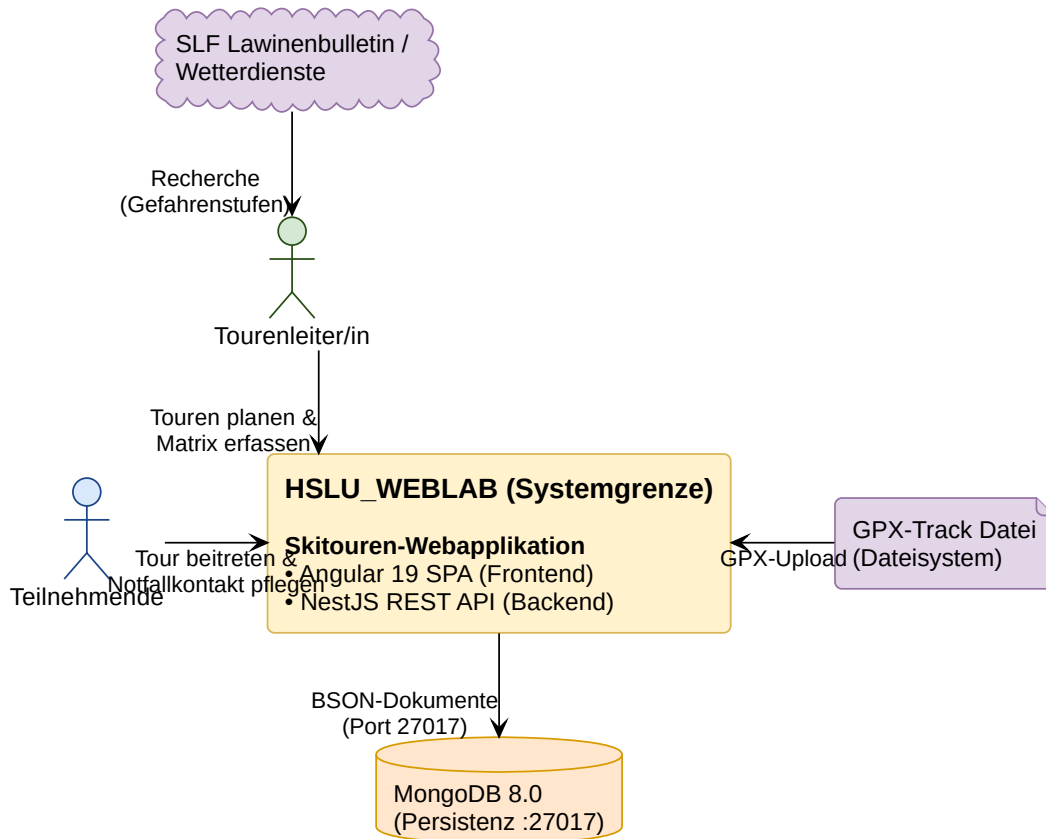


Figure 1: Systemkontext: Benutzer, Schnittstellen und Umsysteme (PlantUML: 01_kontext_abgrenzung.puml)

2.2 Technischer Kontext

- Der Browser kommuniziert über HTTP (Port 80) mit Nginx.
- Nginx leitet API-Aufrufe (/api/*) intern an das NestJS-Backend (Port 4566) weiter.
- Das Backend greift über das MongoDB-Wire-Protokoll (Port 27017) auf die Datenbank zu.

2.3 In Scope

- + **Touren-CRUD:** Erstellen, Bearbeiten, Löschen und Detailansicht von Touren.
- + **Digitale 3x3-Sicherheitsmatrix:** Strukturierte Risikobeurteilung nach Werner Munter (Verhältnisse, Gelände, Mensch).
- + **Notfallkontakt-Erfassung:** Hinterlegung bei der Registrierung und Bearbeitung im Profil.
- + **Geschützter Notfallkontakt-Zugriff:** Zugriff auf Notfalldaten ausschliesslich für autorisierte Tourenleiter der jeweiligen Tour.
- + **Teilnehmer- und Rollenverwaltung:** Beitritt per Link und Verwaltung von Leiter- und Teilnehmerrollen.
- + **GPX-Routenupload:** Hochladen und Anzeigen von GPX-Tracks direkt in der Tour.

2.4 Out of Scope

- **Native Mobile Apps:** Keine eigenständigen Apps für iOS oder Android (reine responsive Web-App).
- **Automatisches Lawinenbulletin:** Kein API-Scraping von SLF- oder Wetterdiensten (manuelle Erfassung).
- **E-Mail-Dienste:** Keine E-Mail-Verifikation und kein Passwort-Reset per Mail.
- **Live-GPS-Tracking:** Keine Turn-by-Turn-Navigation oder Live-Ortung während der Tour.

3 Solution Strategy

Da wir im Modul WebLab freie Technologiewahl hatten, fasst die folgende Tabelle die gewählten Kerntechnologien und die Gründe dafür kurz und bündig zusammen:

Bereich	Gewählte Technologie	Warum gewählt? (Begründung)	ADR
Backend	NestJS 11 (TypeScript)	Klare Schichtenarchitektur (Controller, Service, Modul). Gleiche Denkweise wie Angular und automatische Validierung via <code>class-validator</code> .	ADR-01
Frontend	Angular 19 (Standalone & Signals)	Alles aus einer Hand (Routing, Forms, HTTP). Standalone Components und Signals sparen Modul-Boilerplate und bieten direkte Reaktivität.	ADR-02
Datenbank	MongoDB 8.0 & Mongoose	Hierarchische 3x3-Matrix und Notfallkontakte direkt als BSON einbettbar; keine zeitraubenden SQL-Joins für dynamische Matrixfelder.	ADR-03
Auth	@nestjs/jwt, Passport & bcrypt	Standard-Stack im NestJS-Ökosystem: Sicheres Passwort-Hashing via bcrypt und standardisierte JWT-Verwaltung mit Passport ohne fehleranfällige Eigenbau-Krypto.	ADR-04
Deployment	Docker Compose & Nginx	1-Befehl-Start (<code>docker compose up --build</code>). Nginx serviert das Frontend und leitet <code>/api/</code> weiter; keine CORS-Probleme.	ADR-05
Architektur	Smart / Dumb Components	Smart Container steuern State und API-Calls, Dumb Components zeigen nur an (@Input) und melden Events (@Output).	ADR-02

4 Building Block View

4.1 Bausteinsicht Level 1: Gesamtsystem Whitebox

Das System läuft vollständig isoliert in Docker Compose:

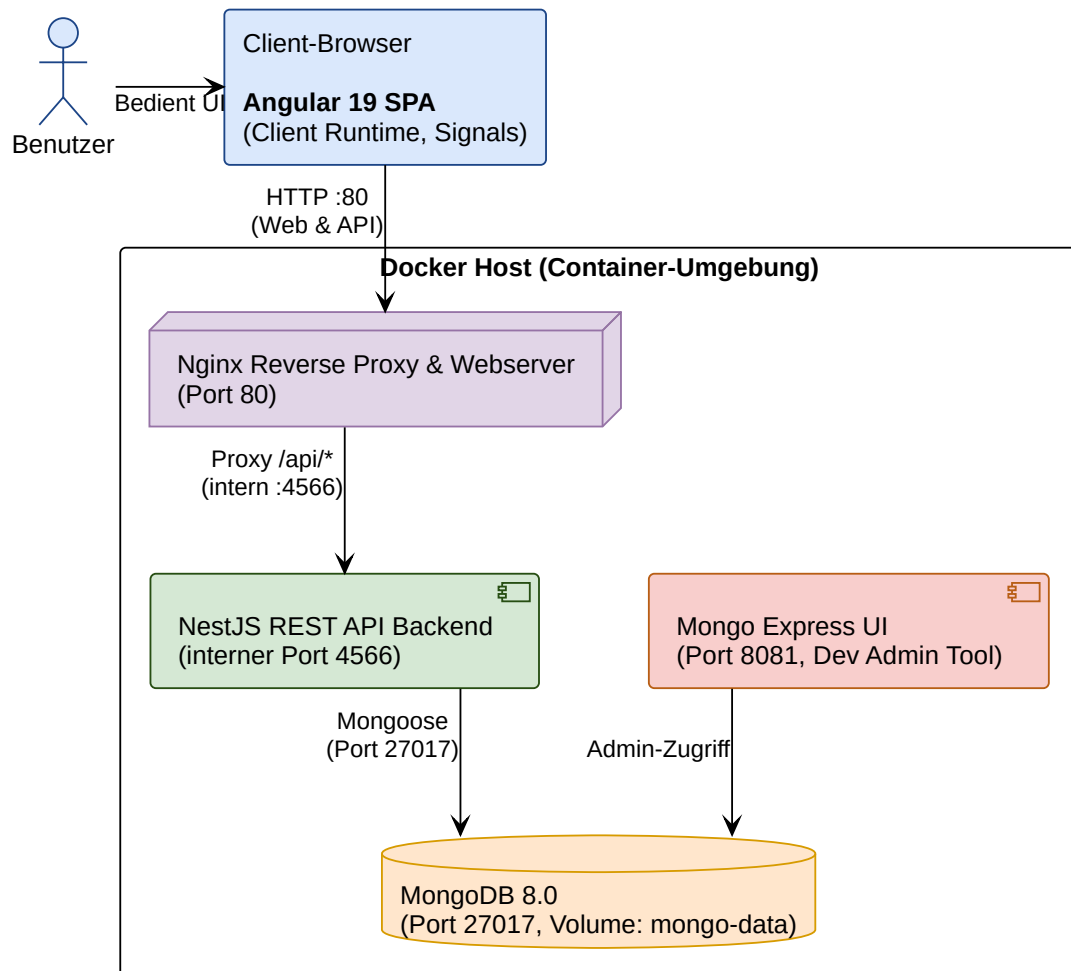


Figure 2: Bausteinsicht Level 1: System-Whitebox mit Containern und Ports (PlantUML: 02_bausteinsicht_level1.puml)

Container	Technologie	Verantwortung & Schnittstellen
HSLU_WEBLAB_frontend	nginx:alpine	Haupteinstiegspunkt (Port 80). Liefert das Angular-Bundle aus und proxied /api/* an das Backend weiter.
backend	NestJS 11, Node.js 24	REST-API auf internem Port 4566. Verarbeitet Business-Logik, Validierung und Berechtigungsprüfungen.
hslu_weblab_mongo	mongo:8	MongoDB-Server auf Port 27017. Speichert Daten dauerhaft im Volume mongo-data.
hslu_weblab_mongo_express	mongo-express:1	Web-Admin-Tool auf Port 8081 zur Datenbank-Inspektion während der Entwicklung.

4.2 Bausteinsicht Level 2: Frontend-Architektur

Das Frontend ist nach Features und dem Smart/Dumb-Muster strukturiert:

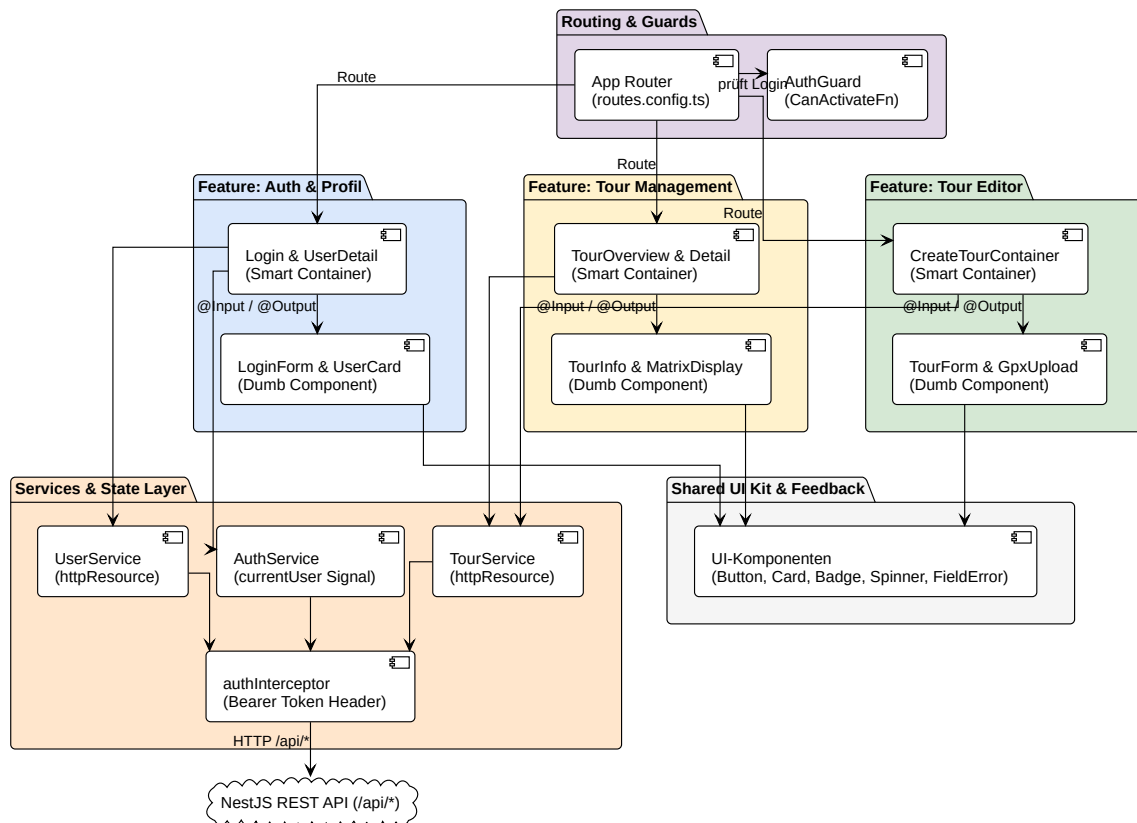


Figure 3: Bausteinsicht Level 2: Frontend-Aufbau mit Smart Containern und Dumb Components (PlantUML: 03_frontend_architektur.puml)

Ordner / Bereich	Inhalt & Verantwortung
features/tour-management/	Tourenübersicht, Detailansicht, Teilnehmerliste, 3x3-Matrix-Anzeige (TourInformation, SecurityMatrixDisplay).
features/tour-editor/	Formulare zur Tourenerstellung und Bearbeitung der 3x3-Matrix, GPX-Upload (TourForm, SecurityMatrixForm).
features/user/	Benutzerprofil, Bearbeitung der Notfallkontakte, geschützte Notfallansicht (UserCard, EmergInformation).
features/auth/	Login- und Registrierungsformulare, AuthService, authInterceptor, authGuard.
components/	Wiederverwendbares UI-Kit (Button, Card, Badge, LoadingSpinner, InputFieldError).
pipes/	Wiederverwendbare Hilfs-Pipes (ShortenerPipe).

4.3 Bausteinsicht Level 2: Backend-Architektur

Das Backend ist als modularer Monolith organisiert:

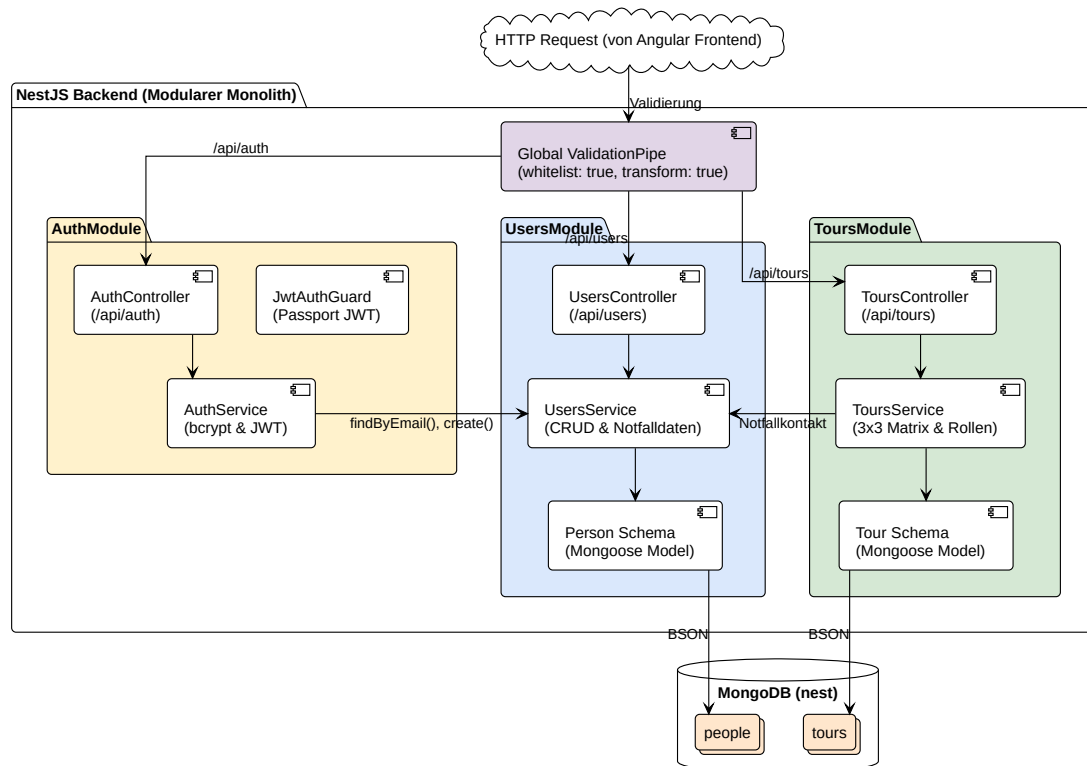


Figure 4: Bausteinsicht Level 2: NestJS Modular Monolith Struktur (PlantUML: 04_backend_architektur.puml)

- **AppModule:** Wurzelmodul, initialisiert MongoDB/Mongoose, globalen ValidationPipe und Telemetry.
- **AuthModule:** Login, Registrierung, Passwort-Hashing (bcrypt) sowie Token-Erstellung via @nestjs/jwt und Absicherung via JwtAuthGuard (@nestjs/passport).
- **UsersModule:** Profilverwaltung; blendet sensible Notfallkontakte bei Standardabfragen immer aus (.select('-emergencyContact')).
- **ToursModule:** Touren, 3x3-Sicherheitsmatrix, Rollen (admin/participant) und autorisierte Notfallkontakt-Abfrage.

5 Runtime View

5.1 Szenario 1: Registrierung & Login

Ablauf bei der Registrierung eines neuen Benutzers mit Notfallkontakt:

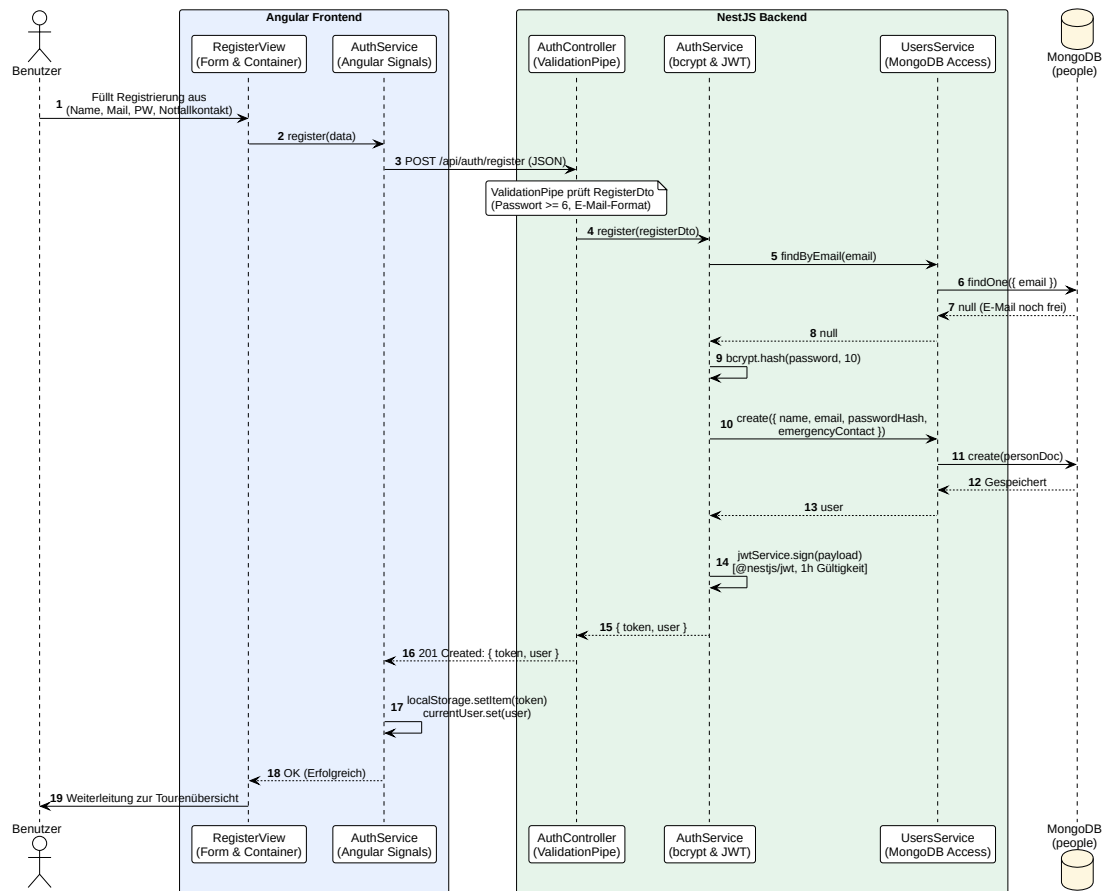


Figure 5: Laufzeitsicht: Registrierung und Token-Erstellung (PlantUML: 05_laufzeitsicht_auth.puml)

1. Benutzer füllt Formular aus (Name, E-Mail, Passwort, Notfallkontakt) und klickt "Konto erstellen".
2. ValidationPipe prüft das RegisterDto (E-Mail-Format, Passwort ≥ 6 Zeichen).
3. Backend hasht das Passwort mit Salt via bcrypt (10 Salt-Rounds) und speichert die Person in MongoDB.
4. Backend signiert ein JWT via JwtService (@nestjs/jwt, 1 Stunde gültig) und schickt es zurück.
5. Frontend speichert das Token im localStorage und leitet zur Tourenübersicht weiter.

5.2 Szenario 2: Tour erstellen mit 3x3 Sicherheitsmatrix

Ablauf beim Planen und Abspeichern einer Tour:

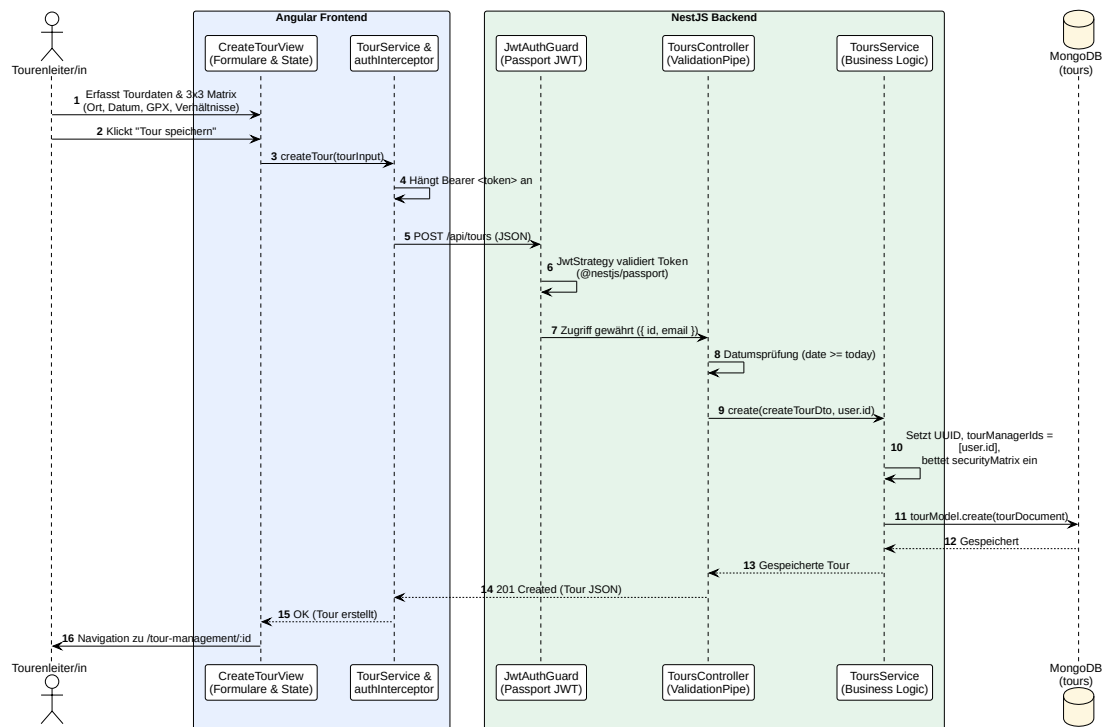


Figure 6: Laufzeitsicht: Tourenerstellung inkl. 3x3-Matrix (PlantUML: 06_laufzeitsicht_tour_erstellung.puml)

1. Tourenleiter erfasst Tourdaten, GPX-Track und füllt die 3x3-Matrix (Lawinstufe, Wind, Gefahren) aus.
2. authInterceptor hängt das JWT als Bearer-Token an den POST-Request an /api/tours.
3. JwtAuthGuard verifiziert das Token und stellt den User bereit.
4. Controller prüft, dass das Datum nicht in der Vergangenheit liegt (`date >= today`).
5. Service trägt den User in `tourManagerIds` ein und speichert die Tour mit der Matrix in MongoDB.
6. Frontend leitet direkt auf die neue Detailansicht weiter.

5.3 Szenario 3: Tour-Beitritt & Notfallkontakt-Abfrage

Ablauf für Beitritt und Berechtigungsprüfung im Notfall:

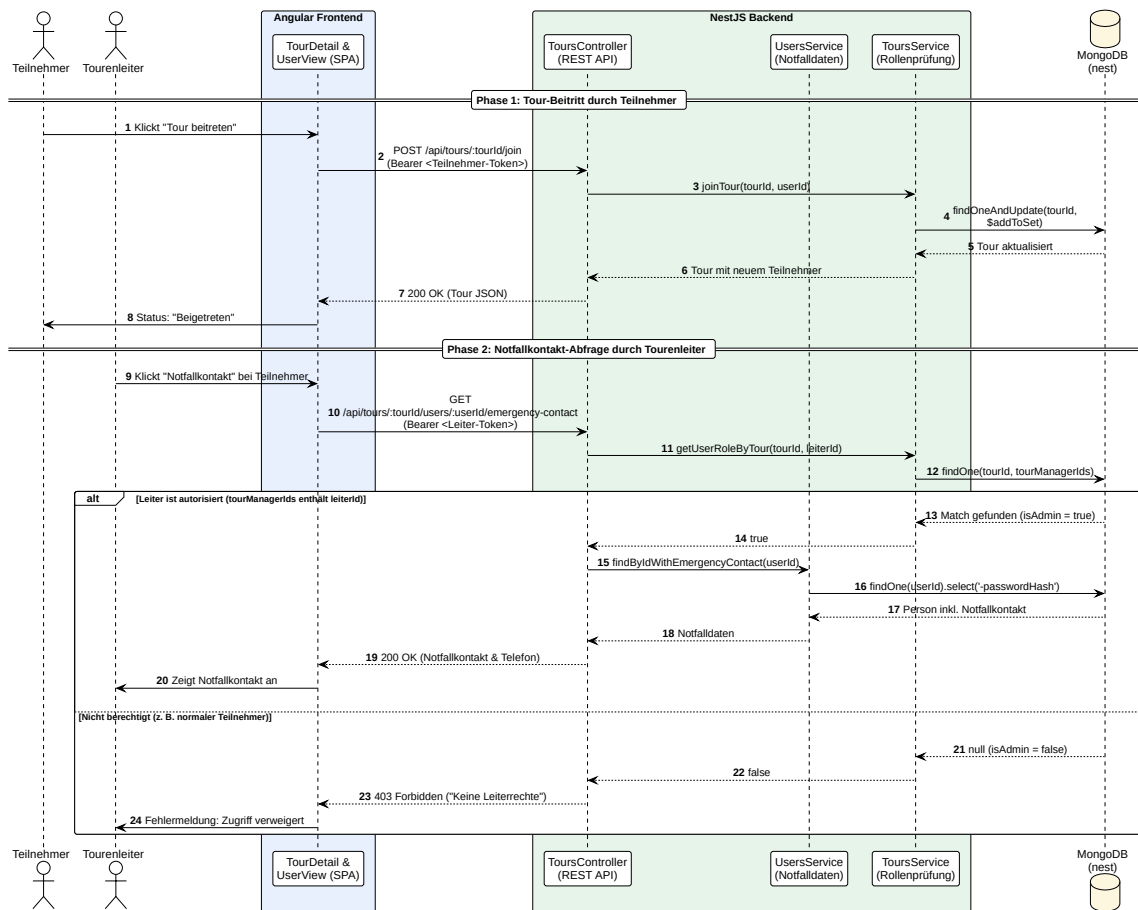


Figure 7: Laufzeitsicht: Tour-Beitritt und Berechtigungsprüfung (PlantUML: 07_laufzeitsicht_notfallkontakt.puml)

1. **Beitritt:** Teilnehmer klickt "Tour beitreten". Backend fügt seine ID atomar via \$addToSet in participantIds ein.
2. **Notfallkontakt-Abruf:** Klickt der Tourenleiter auf einen Notfallkontakt, prüft das Backend mit getUserRoleByTour():
 - **Ist Tourenleiter:** Gibt Notfallkontakt mit HTTP 200 zurück.
 - **Kein Tourenleiter:** Bricht sofort ab mit HTTP 403 Forbidden.

6 Deployment View

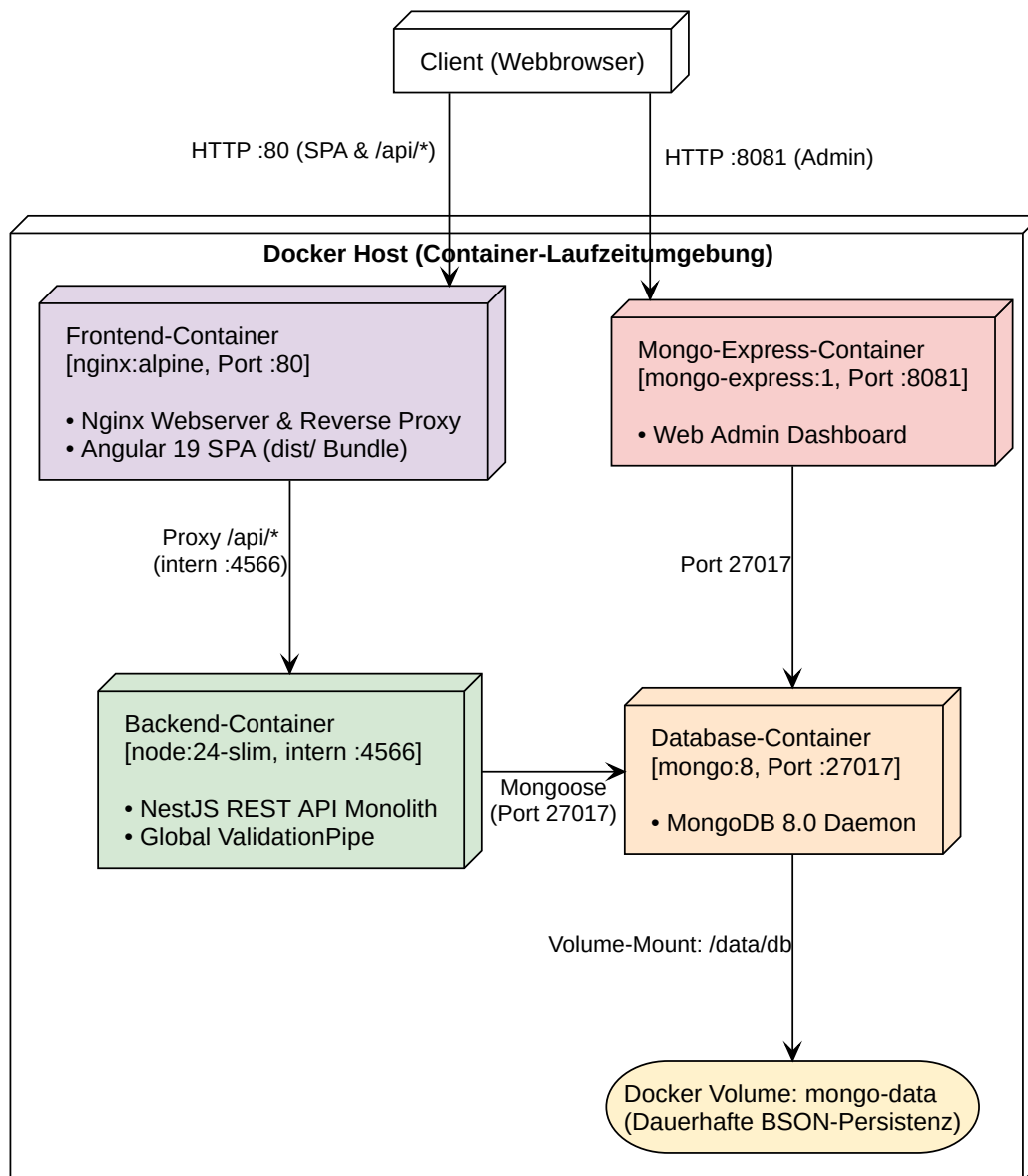


Figure 8: Verteilungssicht: Docker Compose Umgebung (PlantUML: 08_verteilungssicht_deployment.puml)

6.1 Container & Multi-Stage Builds

- **Frontend:** Stage 1 (node:24-slim) baut Angular (npm run build). Stage 2 (nginx:alpine) übernimmt nur den fertigen dist/-Ordner.
- **Backend:** Stage 1 kompiliert TypeScript. Stage 2 führt nur dist/ mit Produktions-Dependencies aus.
- **Nginx-Routing:** Serviert Angular per HTML5-Fallback (try_files \$uri /index.html) und leitet /api/ an das Backend weiter (proxy_pass http://backend:4566). Kein CORS nötig.

7 Crosscutting Concepts

7.1 Ownership, Datenschutz & Sicherheit

- **Datenschutz:** Standardabfragen filtern Notfallkontakte immer serverseitig aus (`.select(' - emergencyContact')`).
- **Authentifizierung:** Zustandsloses JWT, signiert via `@nestjs/jwt` (1h Lebensdauer) und validiert über eine Passport-JWT-Strategie (`JwtStrategy`). Passwörter werden sicher mit `bcrypt` gehasht (10 Salt-Rounds) und verifiziert.
- **Validierung & NoSQL-Schutz:** Der globale `ValidationPipe` (`whitelist: true, transform: true`) verwirft unbekannte Payload-Felder. Falsche Datumsformate oder NoSQL-Injection-Versuche (z. B. `$gt`, `$where`) werden sofort mit HTTP 400 geblockt, bevor sie die Datenbank erreichen.
- **Fehlerformat (Error Handling):** NestJS liefert Fehler standardisiert als JSON: `{ "statusCode": 400, "message": [ "..."], "error": "Bad Request" }`. Das Frontend wertet `error.message` aus und zeigt sie im UI an. Bei HTTP 401 loggt der `authInterceptor` automatisch aus.

7.2 Datenmodell & Konsistenz

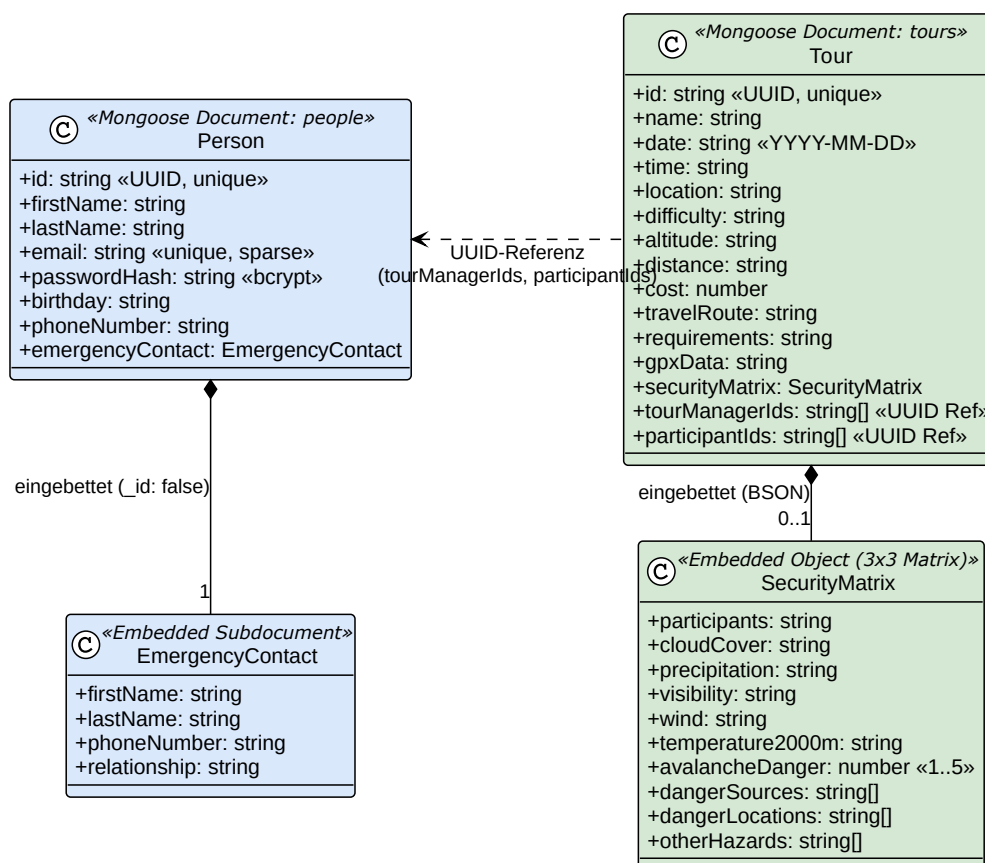


Figure 9: Datenmodell: Mongoose Schemas und Beziehungen (PlantUML: 09_datenmodell_persistenz.puml)

- **Eingebettet:** EmergencyContact in Person (`_id: false`) und SecurityMatrix in Tour.
- **UUIDs statt MongoDB ObjectIds:** Aus Gewohnheit aus früheren Projekten habe ich IDs manuell als UUIDs (`crypto.randomUUID()`) generiert und Referenzen im Service-Layer per `findByIds({ id: { $in: ids } })` statt via `Mongoose-populate()` aufgelöst. Da dies stabil lief und die Domäne sauber von Datenbank-Interna entkoppelt, habe ich es so beibehalten.

7.3 Frontend-Konzepte & Reaktivität

Thema	Umsetzung & Ansatz
Reaktivität	Angular Signals (signal, computed) für lokalen State; httpResource für deklaratives Laden.
Komponenten-Kommunikation	Smart Container verwalten State; Dumb Components nutzen ausschließlich @Input und @Output.
Change Detection	ChangeDetectionStrategy.OnPush auf allen Komponenten für gute Performance.
Styling & Feedback	CSS Design-System mit Farbcodes für Lawinenwarnstufen; Validierungsfeedback via InputFieldError.
Offline-Strategie	Tokens/User im localStorage. Für Notfallkontakte ohne Netz ist der Ausbau zur PWA konzipiert.

7.4 Testing

Test-Typ	Werkzeug	Anzahl	Was wird abgedeckt?
Backend Unit	Vitest	9 Files	Services (users, tours, auth), JwtStrategy, JwtAuthGuard, DTO-Validierung, Hash-Utills.
Backend Integration	Vitest / Supertest	1 File	REST-Endpunkte gegen die Test-Datenbank (api.integration.spec.ts).
Frontend Unit	Vitest	23 Files	Smart Container, Dumb Components (security-matrix), Services, Pipes, UI-Kit.
Frontend E2E	Cypress	5 Files	Auth-Flow, Tourenverwaltung, Toureneditor, Tourdetails, Navigation.

8 Architekturentscheidungen (ADRs)

ADR	Thema	Entscheidung	Begründung
ADR-01	Backend	NestJS 11 (Type-Script)	Klare Schichten, Dependency Injection, fühlt sich wie Angular an und spart Validierungscode via <code>class-validator</code> .
ADR-02	Frontend	Angular 19 Stand-alone	Alles aus einer Hand (Routing, Forms, HTTP), kein altes Modul-Boilerplate, saubere Trennung durch Smart/Dumb-Muster.
ADR-03	Datenbank	MongoDB & Mongoose	3x3-Matrix und Notfallkontakte lassen sich direkt als Dokument einbetten, ohne zeitraubende SQL-Joins und Tabellen.
ADR-04	Auth	@nestjs/jwt & bcrypt	Standard-Ökosystem in NestJS: Etablierte JWT-Strategie mit Passport und robustes Bcrypt-Hashing statt wartungsintensiver Eigenbau-Kryptografie.
ADR-05	Deployment	Docker Compose & Nginx	1-Befehl-Start (<code>docker compose up --build</code>); Nginx verhindert CORS-Probleme, da alles über Port 80 läuft.
ADR-06	Datenschutz	Restriktive Notfallkontakte	Principle of Least Privilege: Notfallkontakte sind nur für den verifizierten Tourenleiter der jeweiligen Tour sichtbar.

9 Qualitätsanforderungen

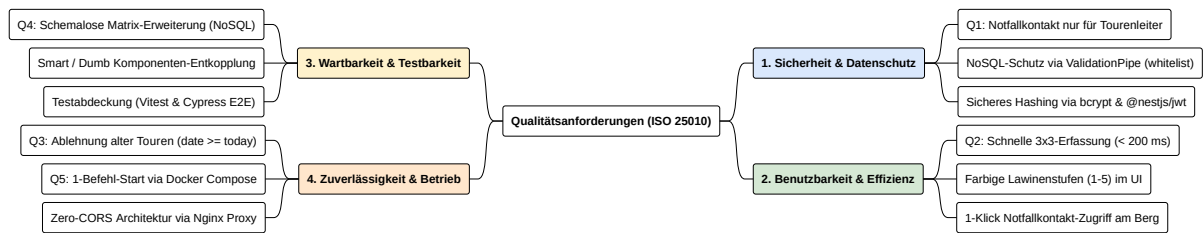


Figure 10: Qualitätsbaum: Gliederung der Qualitätsziele nach ISO 25010 (PlantUML: 10_qualitaetsbaum.puml)

ID	Szenario	Verhalten des Systems	Messkriterium
Q1	Teilnehmer ruft Notfall-URL eines anderen Teilnehmers auf.	Backend prüft Leiterrechte und blockiert den Zugriff.	HTTP 403 Forbidden.
Q2	Tourenleiter speichert 3x3-Sicherheitsmatrix ab.	Formular validiert Daten; Matrix wird sofort gespeichert und gerendert.	Ladezeit < 200 ms lokal.
Q3	Beitrittsversuch zu einer vergangenen Tour.	Backend prüft date < today und lehnt den Beitritt ab.	HTTP 400 Bad Request.
Q4	Matrix wird später um neue Gefahrenfelder erweitert.	Da MongoDB schemalos speichert, muss nur das DTO angepasst werden.	Keine SQL-Migration nötig.
Q5	Start auf einem frischen Entwicklungsrechner mit Docker.	Startet Frontend, Backend und Datenbank ohne manuelle Vorarbeit.	docker compose up auf Port 80.

10 Risiken & technische Schulden

Bereich	Problem / Risiko	Status & Massnahme
Offline-Nutzung am Berg	Im alpinen Funkloch schlägt das Neuladen fehl; Notfallkontakte wären offline nicht erreichbar.	Konzipiert als nächster Schritt: PWA mit Service Worker und IndexedDB-Cache für Notfallnummern.
GPX-Parsing im Browser	Riesige GPX-Dateien (> 20 MB) könnten den UI-Thread kurz blockieren.	Aktuell für normale GPX-Tracks ausreichend; künftig Auslagerung in einen Web Worker.
E2E-Testisolation	Cypress-Tests teilen sich teilweise Datenbankzustände.	Tests bereinigen ihre Daten eigenständig; für Einzelentwickler im Modul WebLab vollkommen ausreichend.